Week 21.5

2303A52392

BATCH 43

TASK A:

PROMPT: Create a simple Python program to calculate student grades based on marks.

Code:

week21.5.py > ...

```python
# Basic version
marks = list(map(int, input("Enter marks: ").split()))
avg = sum(marks) / len(marks)

if avg >= 90:
    print("Grade: A")
elif avg >= 75:
    print("Grade: B")
elif avg >= 50:
    print("Grade: C")
else:
    print("Grade: Fail")

# Enhanced version
def calculate_grade(marks):
    avg = sum(marks) / len(marks)
    if avg >= 90:
        return "A"
    elif avg >= 75:
        return "B"
    elif avg >= 50:
        return "C"
    else:
        return "Fail"
def get_marks():
```

```python
def get_marks():
    try:
        marks = list(map(int, input("Enter marks: ").split()))
        if any(m < 0 or m > 100 for m in marks):
            raise ValueError("Marks should be between 0 and 100")
        return marks
    except:
        print("Invalid input. Try again.")
        return get_marks()
marks = get_marks()
print("Grade:", calculate_grade(marks))
```

Output:

```
Could not find platform independent libraries <prefix>
Enter marks: 20
Grade: Fail
Enter marks: 60
Grade: C
(.venv) PS C:\Users\vikas\Downloads\AI Assist Coding>
```

Summary:

AI helps generate quick initial solutions and improves them with validation and features. Enhanced versions are more user-friendly, reliable, and scalable.

TASK B:

PROMPT: Create a simple library management system in Python.

Code:

```python
#Task-2
books = []
def add_book(book):
    books.append(book)
def show_books():
    print("Books:", books)
add_book("Python")
add_book("Java")
show_books()

#More advanced version
library = {}
def add_book(title, quantity):
    library[title] = library.get(title, 0) + quantity
def issue_book(title):
    if title in library and library[title] > 0:
        library[title] -= 1
        print(f"{title} issued")
    else:
        print("Book not available")
def return_book(title):
    library[title] = library.get(title, 0) + 1
def show_books():
    for book, qty in library.items():
        print(f"{book}: {qty}")
add_book("Python", 3)
issue_book("Python")
return_book("Python")
show_books()
```

Output:

```
Books: ['Python', 'Java']
Python issued
Python: 3
(.venv) PS C:\Users\vikas\Downloads\AI Assist Coding>
```

Summary: AI enhances basic programs by adding modular structure and features. Improved applications are more interactive and closer to real-world systems.

TASK C:

PROMPT: Git Workflow Practice

```
#task 3
#Task-3
# Initialize repo
git init
# Add files
git add .
# Commit
git commit -m "Initial commit - basic program"
# Create branch
git branch feature-update
# Switch branch
git checkout feature-update
# Make changes and commit
git commit -am "Added enhanced features"
# Merge back
git checkout main
git merge feature-update
```

Summary:

Git helps manage versions and track AI-generated changes effectively.
Branching allows safe experimentation before merging into the main project.

TASK G:

PROMPT: IMPLEMENTCOLLABORATIVE DEVELOPMENT

CODE:

```
library = {}

# Student A feature
def add_book(title, qty):
    library[title] = library.get(title, 0) + qty

# Student B feature (AI assisted)
def issue_book(title):
    if library.get(title, 0) > 0:
        library[title] -= 1
        print(f"{title} issued")
    else:
        print("Book not available")

def return_book(title):
    library[title] = library.get(title, 0) + 1
```

OUTPUT:

```
Books: ['Python', 'Java']
Python issued
Python: 3
```

SUMMARY: Collaborative development with Git branches allows multiple contributors to work independently and merge efficiently.
AI enhances productivity by quickly generating features and improving code quality.

TASK F:

Prompt: Improve the given Python function for addition by adding input validation and error handling so it can safely handle invalid or non-numeric inputs.

```python
    # TASK F - PAIR PROGRAMMING
# Student A code
def add(a, b):
    return a + b
# Student B (AI improved version)
def add_safe(a, b):
    try:
        return float(a) + float(b)
    except:
        return "Invalid input"
if __name__ == "__main__":
    print(add_safe(5, 10))  # Output: 15.0
    print(add_safe("5", "10"))  # Output: 15.0
    print(add_safe("five", "ten"))  # Output: Invalid input
```

Output:

```
15.0
15.0
Invalid input
(.venv) PS C:\Users\vikas\Downloads\AI Assist Coding>
```

Summary:

Pair programming combined with AI helps improve code quality by adding validation and robustness. AI-assisted enhancements make programs safer and more reliable for real-world usage.